High-Performance Systems Programming

תרגול 1: מבוא לשפת C



תרגולים ויצירת קשר

- מתרגל :עידן גולדברג
- דוא"ל goldidan@post.bgu.ac.il :

תרגולים :יום א'

- 15:00 - 16:00
- 16:00 - 17:00
- 17:00 - 18:00

- שעות קבלה :יום שני12:00 - 14:00 ,
- **יש לתאם מראש.**

תרגולים ויצירת קשר

- מתרגל :נור עתאמנה
- דוא"ל: atamni@post.bgu.ac.il

תרגולים ביום שני:

- 17:00–18:00
- 18:00–19:00
- 19:00–20:00

- שעות קבלה :יום שני 15:00–17:00 ,
- **חובה לתאם מראש במייל.**

סביבת עבודה

- העבודה בקורס תתבצע בסביבת **Linux (חובה.)**
- למשתמשי Windows: ניתן לעבוד באמצעות WSL או להתקין Linux לצד Windows (Dual Boot).
- מומלץ להשתמש ב־ **Visual Studio Code** לכתיבת הקוד.
- יש לוודא שמהדר GCC מותקן בסביבת Linux, **שניתן לקמפל ולהריץ בה תוכנית C פשוטה**, למשל Hello World.
- ניתן לעבוד גם בסביבת Linux במחשבי המעבדות של האוניברסיטה.

Anatomy of a C Program

```c
#include <stdio.h>

    int main(void) {

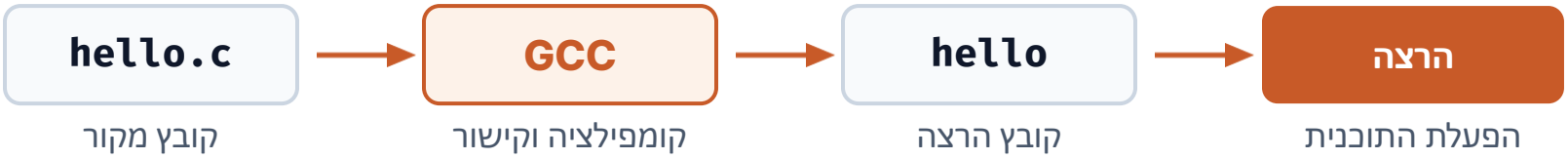
    printf("Hello, world!\n");

    return 0;

}
```

- `#include <stdio.h>` → Declares standard I/O functions
- `int main(void)` → Program entry point
- `printf(...)` → Writes to standard output
- `return 0` → Reports successful completion

קומפילציה והרצה



פקודות בטרמינל

```
$ gcc -std=c11 -Wall -Wextra hello.c -o hello
$ ./hello
```

פלט

```
Hello, world!
```

שימוש בתקן C11	`-std=c11`
הפעלת אזהרות נוספות	`-Wall -Wextra`
שם קובץ ההרצה	`-o hello`

משתנים וטיפוסי מידע

לכל משתנה יש טיפוס שקובע ביצד הערך מיוצג ואילו ערכים ניתן לאחסן בו.




```
int age;          // הגדרה ללא אתחול
int age = 23;     // הגדרה ואתחול
age = 25;         // השמת ערך
double average = 87.5;   char grade = 'A';
```

אזהרה: משתנה מקומי שלא אותחל אינו מקבל אוטומטית את הערך 0. יש לתת לו ערך לפני קריאתו.

טיפוסי מידע בסיסיים

לכל טיפוס מידע יש שימוש וגודל בזיכרון.

טיפוס	מה הוא מייצג?	C-דוגמה ב	גודל נפוץ
`char`	תו בודד	`char grade = 'A';`	1 Byte
`int`	מספר שלם	`int age = 23;`	4 Bytes
`float`	מספר עשרוני	`float x = 3.14f;`	4 Bytes
`double`	מספר עשרוני בדיוק גבוה	`double pi = 3.14159;`	8 Bytes

הערה: הגדלים המוצגים הם גדלים נפוצים .הגודל בפועל עשוי להשתנות בהתאם למערכת ולמהדר.

```
printf("%zu", sizeof(int));
```

האופרטור `sizeof` מאפשר לבדוק את הגודל של טיפוס או משתנה בבתים.

קבועים Define & Const :

const

הגדרת משתנה בעל טיפוס ,שלא ניתן לשנות את ערכו באמצעות השמה.

```
const int limit = 30;
limit = 40; // Error!
```

ניסיון לשנות את הערך גורם לשגיאת קומפילציה.

define#

הגדרת מאקרו :קדם־המעבד מחליף את שם המאקרו בתוכן שהוגדר.

```
#define MAX 30
int count = MAX;
```

לאחר הרחבת המאקרו:

```
int count = MAX; → int count = 30;
```

הערה :בדוגמת const מסיימים את ההגדרה ב־ ;ואילו בהגדרת המאקרו כאן אין להוסיף ;

קלט ופלט

הפונקציה printf: פלט מעוצב

הפונקציה printf מדפיסה פלט מעוצב ל־standard output.

דוגמת קוד ב־C:

```c
#include <stdio.h>
int age = 23;
printf("Age: %d\n", age);
```

הפלט המתקבל (Output):

```
Age: 23
```

ניתוח רכיבי הפקודה printf("Age: %d\n", age);

"Age: "

טקסט מחרוזת שמודפס כפי שהוא בדיוק.

%d / \n

%d – מציין המרה (format specifier) המורה להדפיס ערך של מספר שלם (int).

\n – תו מיוחד למעבר שורה חדשה.

age

המשתנה שממנו נלקח הערך שיודפס במקום ה־%d.

הערה: הקובץ stdio.h מכיל את ההצהרה של הפונקציה printf ולכן יש לכלול אותו בתחילת התוכנית עם #include).

פורמטים להדפסה ב־printf()

כיצד בוחרים את מציין ההמרה המתאים?

מציין	מה הוא מציג?	דוגמת פלט
%d	מספר שלם	42
%f	מספר עשרוני)ברירת מחדל 6 :ספרות(	3.140000
%c	תו בודד	A
%s	מחרוזת	Hello

דוגמת שימוש בקוד:

```
printf("Age: %d | Grade: %.2f", 21, 94.75);
Output: Age: 21 | Grade: 94.75
```

כל מציין המרה קובע כיצד printf() מציגה את הערך.

)המציין %f מדפיס 6 ספרות כברירת מחדל; בדוגמה השתמשנו ב־ %.2f בדי לשלוט בדיוק – כפי שנלמד בשקף הבא(

הדפסת שלמים

```
#include <stdio.h>
void main() {
    int x = -246;
    printf("Decimal: %d\nUnsigned: %u\n", x, x);
    printf("Hex: %X\nhex: %x\nOctal: %o\n", x, x, x);
}
```

Output:

הדפסת שלמים

```
#include <stdio.h>
void main() {
    int x = -246;
    printf("Decimal: %d\nUnsigned: %u\n", x, x);
    printf("Hex: %X\nhex: %x\nOctal: %o\n", x, x, x);
}
```

Output:

```
Decimal: -246
Unsigned: 4294967050
Hex: FFFFFF0A
hex: ffffff0a
Octal: 37777777412
```

הדפסת עשרוניים

```
#include <stdio.h>
void main() {
    float f = 251.733733733;
    printf("%f\n%.8f\n%.3f\n%e\n%E\n", f, f, f, f, f);
}
```

Output:

הדפסת עשרוניים

```
#include <stdio.h>
void main() {
    float f = 251.733733733;
    printf("%f\n%.8f\n%.3f\n%e\n%E\n", f, f, f, f, f);
}
```

Output:

```
251.733734
251.73373413
251.734
2.517337e+02
2.517337E+02
```

הדפסת תווים

```c
#include <stdio.h>

void main() {
    // Don’t use magic numbers:
    const int caseDiff = 'A' - 'a'; // -32

    char c = 'a';
    printf("Char: %c\nValue: %d\n\n", c, c);
    c = c + 1;
    printf("Char: %c\nValue: %d\n\n", c, c);
    c = c + caseDiff;
    printf("Char: %c\nValue: %d\n", c, c);
}
```

Output:

Char: a
Value: 97

Char: b
Value: 98

Char: B
Value: 66

הדפסת כתובות

```
#include <stdio.h>

void main() {
    char c = 'h';
    int x = -9234;
    float f = 251.7366;
    printf("x: %p\nf: %p\nc: %p\n", &x, &f, &c);
}
```

Output:

```
c: 0x7fffc91ed10a
x: 0x7fffc91ed10b
f: 0x7fffc91ed10f
```

הפונקציה scanf: קריאת קלט

הפונקציה scanf קוראת קלט וממירה אותו לפי מחרוזת הפורמט.

הסבר הרכיבים:

- `%d` – מציין המרה שמורה ל־scanf לקרוא מספר שלם.
- `&age` – הכתובת שבה scanf תשמור את הערך שנקלט.

דוגמת קוד ב־C:

```c
#include <stdio.h>
int age;
scanf("%d", &age);
```

זרימת הקליטה בזיכרון:

1. **קלט מהמשתמש**
   `21`
   המשתמש מקליד ערך בטרמינל

→

2. **ביצוע הפקודה**
   `scanf("%d", &age);a`
   המרת מחרוזת למספר שלם

→

3. **שמירה בזיכרון**
   `age = 21`
   מצב המשתנה לאחר הקריאה

דגש חשוב:
הפונקציה **scanf** שמורה לשנות את תוכן המשתנה בזיכרון ישירות דרך כתובתו (**&age**), ואינה מייצרת פלט למסך.

פורמטים לקליטת נתונים ב־scanf()

כיצד בוחרים את מציין ההמרה המתאים?

מציין	מה הוא קולט?	דוגמת קלט
%d	מספר שלם	42
%f	מספר עשרוני בדיוק יחיד (float)	3.14
%lf	מספר עשרוני בדיוק כפול (double)	3.14159
%c	תו בודד	A
%s / %19s	מחרוזת תווים	Hello

דוגמת שימוש בקוד:

```
int age;  double grade;
scanf("%d %lf", &age, &grade);
```

```
Input:  21 94.75
```
(הקלט מופרד ברווח או שורה חדשה – age=21, grade=94.75)

- **הבחנה בדיוק:** ב־scanf קולטים float באמצעות %f, אך עבור double חובה להשתמש ב־%lf.
- **העברת כתובת:** משתנים רגילים נדרשים לאופרטור הכתובת (&) למשל &age). במערך תווים (char[]) מעבירים את שם המערך ללא &.
- **הגנת גלישת זיכרון:** עבור מערך char name[20], מומלץ להשתמש ב־%19s להגבלת 19 תווים והשארת מקום לתו המסיים '\0'.

דוגמה 1

```
#include <stdio.h>
void main() {
    int width, height, area;
    printf("Enter width: ");
    scanf("%d", &width);
    printf("Enter height: ");
    scanf("%d", &height);
    area = width * height;
    printf("The area is %d\n", area);
}
```

Output:

Enter width: 5

Enter height: 100

The area is 500

דוגמה 2

```
#include <stdio.h>
void main() {
    int num1, num2;
    printf("Enter 2 numbers: ");
    scanf("%d%d", &num1, &num2);
    printf("The sum is %d\n", num1 + num2);
}
```

Output (1):

Enter 2 numbers: 5 100

The sum is 105

Output (2):

Enter 2 numbers: 5

100

The sum is 105

stdin Buffer

- בביצוע פקודת scanf ריצת התוכנית נעצרת והיא ממתינה שהמשתמש יספק קלט .המשתמש מקליד מספר תווים כלשהו , ומסיים את הפעולה באמצעות הקשה על Enter.

- הקלט שהוזן על ידי המשתמש ממתין בעת בתוך) bufferשנקרא STDIN – Standard Input), והפונקציה scanf מרוקנת אותו בשלבים במהלך ה.parsing-

- במידה ונשארו תווים ב buffer-ריצת התוכנית לא תיעצר בביצוע הפקודה ,והפונקציה תשתמש בתווים הקיימים לפני שהיא פונה אל המשתמש בדי שיספק קלט נוסף.

דוגמה 3

```
#include <stdio.h>
void main() {
    int x;
    char c;
    scanf("%d", &x);

    scanf("%c", &c);

}
```

Output:
1

דוגמה 3

```
#include <stdio.h>
void main() {
    int x;
    char c;
    scanf("%d", &x);
    printf("|%d|\n", x);
    scanf("%c", &c);
    printf("|%c|\n", c);
}
```

Output:

```
1
|1|
|
|
```

פונקציה להצגת stdin

```
#include <stdio.h>
void showBuffer() {
    static int count = 0;
    char* p = stdin->_IO_read_ptr;
    char* end = stdin->_IO_read_end;
    printf("Call %d: |", count++);
    while (p != end) {
        putchar(*p++);
    }
    printf("|\n");
}
```

אין צורך להבין אותה כרגע...

שימוש בפונקציה להצגת STDIN

```
…
void main() {
     char c;
     showBuffer(); // Call 0
     scanf("%c", &c);
     printf("|%c|\n", c);
     showBuffer(); // Call 1
     scanf(" %c", &c);
     printf("|%c|\n", c);
     showBuffer(); // Call 2
}
```

Output:

```
Call 0: ||
 a bcd
| |
Call 1: |a bcd
|
|a|
Call 2: | bcd
|
```

שיפור

```
…
void main() {
     char c;
     showBuffer(); // Call 0
     scanf(" %c", &c);
     printf("|%c|\n", c);
     showBuffer(); // Call 1
     scanf(" %c", &c);
     printf("|%c|\n", c);
     showBuffer(); // Call 2
}
```

Output:

```
Call 0: ||
 a bcd
|a|
Call 1: | bcd
|
|b|
Call 2: |cd
|
```

ניקוי stdin

אפשר לנקות את הבאפר עם שורת הקוד:

```
while (getchar() != '\n');
```

זה נפטר מכל התווים ב stdin-עד וכולל תו שורה חדשה.

getchar():

במו scanfאך זה לוקח רק תו אחד מ.stdin-

דוגמה 5

```c
#include <stdio.h>

int main() {
    int x;
    char ch;

    printf("Enter a number: ");
    scanf("%d", &x);


    printf("Enter a character: ");
    scanf("%c", &ch);  // Reads leftover '\n' from previous input

    printf("x = %d, ch = '%c'\n", x, ch);
    return 0;
}
```

דוגמה 5

```c
#include <stdio.h>

int main() {
    int x;
    char ch;

    printf("Enter a number: ");
    scanf("%d", &x);
    while (getchar() != '\n');  // clear buffer

    printf("Enter a character: ");
    scanf("%c", &ch);  // Now it reads the actual user input

    printf("x = %d, ch = '%c'\n", x, ch);
    return 0;
}
```

תנאי

- המבנה הכללי של תנאי הוא:

```
if (condition)
        statement_1;
else
        statement_2;
```

- אם ערכו של condition הוא "אמת", יתבצע statement_1.
- אם ערכו של condition הוא "שקר", יתבצע statement_2.

- ניתן להגדיר פקודת if ללא else.
- במקום כל statement ניתן להגדיר גם בלוק של פקודות.
- else תמיד מתייחס ל-if האחרון הקרוב ביותר שאין לו else.
- פקודת if נחשבת להוראה יחידה, לכן אפשר לא לתחום אותה בבלוק אם היא מופיעה כחלק מפקודה מורכבת יותר.

דוגמה לתנאי פשוט

```
#include <stdio.h>
void main() {
    int x;
    printf("Enter a number:\n");
    scanf("%d", &x);
    if (x == 0)
        printf("It was a zero!\n");
    printf("It was a number!\n");
}
```

Output:
Enter a number:
10
It was a number!

Output:
Enter a number:
0
It was a zero!
It was a number!

דוגמה לתנאי מקונן – else if

```
#include <stdio.h>
void main() {
    int x1, x2, x3;
    printf("Enter 3 numbers:\n");
    scanf("%d%d%d",&x1, &x2, &x3);

    if (x1 > x2 && x1 > x3) {
        printf("%d", x1);
    } else if (x2 > x1 && x2 > x3) {
        printf("%d", x2);
    } else {
        printf("%d", x3);
    }
    printf(" is the biggest\n");
}
```

Output:

```
Enter 3 numbers:
5
3
8
8 is the biggest
```

לולאת while

- המבנה הכללי של לולאת while הוא:

```
while (condition) {
        statement;
}
```

- כל עוד ערכו של התנאי (condition) הוא "אמת", יתבצע גוף הלולאה (statement).
- ייתכן שהלולאה לא תתבצע כלל.
- אם גוף הלולאה לא משפיע על ערכו של התנאי, תיתכן לולאה אינסופית.
- במקום statement יחיד ניתן להגדיר גם בלוק של פקודות.
- כל פעם ש statement- מתבצע נקראת **איטרציה**.

לולאת for

- המבנה הכללי של לולאת forהוא:

```
for (initialization; condition; progress) {
        statement;
}
```

- ראשית מבוצע האתחול .(initialization)
- כל עוד ערכו של התנאי (condition)הוא "אמת ,"יתבצע גוף הלולאה (statement)ולאחר מכן יתבצע הקידום .(progress)
- ייתכן שהלולאה לא תתבצע כלל)אבל האתחול מתבצע תמיד.(
- אם גוף הלולאה והקידום לא משפיעים על ערכו של התנאי תיתכן לולאה אינסופית.
- במקום statementיחיד ניתן להגדיר גם בלוק של פקודות.

מחרוזות

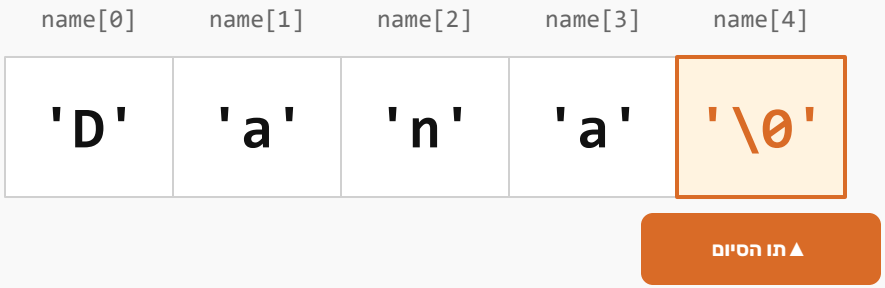
מחרוזות ב־C

מחרוזת ב־C היא רצף תווים שמסתיים בתו `'\0'`. ל־C אין טיפוס מחרוזת מובנה.

הגדרת מחרוזת ואתחולה בקוד:

```c
char name[] = "Dana";
```

ייצוג המערך ותו הסיום בזיכרון (Memory Layout):



הסבר: התוכן כולל 4 תווים; המערך מכיל 5 תאים, כולל תו הסיום.

דגש לקראת ההמשך:

פונקציות מחרוזת נעזרות ב־`'\0'` כדי לזהות היכן המחרוזת מסתיימת.

פונקציות עבודה עם מחרוזות ב־C

ספרית <string.h>

Function	Description	Example
strlen(s)	Returns string length (excluding \0)	strlen("hello") → 5
strcpy(dest, src)	Copies a string	strcpy(dst, "hi") → dst = "hi"
strncpy(dest, src, n)	Copies at most n characters	strncpy(dst, "hi!", 2) → dst = "hi"
strcat(dest, src)	Appends a string	strcat(dst, "!") → dst = "hi!"
strncat(dest, src, n)	Appends at most n characters	strncat(dst, "ya", 1) → dst = "hiy"
strcmp(a, b)	Compares two strings	strcmp("a", "b") → a[i] - b[i]
strncmp(a, b, n)	Compares at most n characters	strncmp("ab", "ac", 1) → 0
strchr(s, c)	Finds first occurrence of a character	strchr("cat", 'a') → "at" (a pointer)
strrchr(s, c)	Finds last occurrence of a character	strrchr("aba", 'a') → "a" (a pointer)
strstr(s, sub)	Finds first occurrence of a substring	strstr("hello", "ll") → "llo"(a pointer)
strtok(s, delim)	Splits a string into tokens	strtok("a,b", ",") → "a"

השוואת מחרוזות

```
int strcmp_impl(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] && (str1[i] == str2[i])) {
        i++;
    }

    return str1[i] - str2[i];
}
```

השוואת מחרוזות - strcmp

```
#include <string.h>
int main() {
    char str[] = "aaa";
    char str1[] = "aaaaa";
    char str2[] = "aab";
    char str3[] = "bbb";
    char str4[] = "aaa";

    printf("%d\n", strcmp(str, str1));
    printf("%d\n", strcmp(str, str2));
    printf("%d\n", strcmp(str, str3));
    printf("%d\n", strcmp(str, str4));

    printf("%d\n", strcmp(str1, str));
    printf("%d\n", strcmp(str2, str));
    printf("%d\n", strcmp(str3, str));
    printf("%d\n", strcmp(str4, str));
}
```

השוואת מחרוזות - strcmp

```
#include <string.h>
int main() {
    char str[] = "aaa";
    char str1[] = "aaaaa";
    char str2[] = "aab";
    char str3[] = "bbb";
    char str4[] = "aaa";

    printf("%d\n", strcmp(str, str1));
    printf("%d\n", strcmp(str, str2));
    printf("%d\n", strcmp(str, str3));
    printf("%d\n", strcmp(str, str4));

    printf("%d\n", strcmp(str1, str));
    printf("%d\n", strcmp(str2, str));
    printf("%d\n", strcmp(str3, str));
    printf("%d\n", strcmp(str4, str));
}
```

Output:

```
-97
-1
-1
0

97
1
1
0
```

השוואת מחרוזות

```
int main() {
    char str[] = "aaa";
    char str1[] = "aaaaa";
    char str2[] = "aab";
    char str3[] = "bbb";
    char str4[] = "aaa";

    printf("%d\n", strcmp_impl(str, str1));
    printf("%d\n", strcmp_impl(str, str2));
    printf("%d\n", strcmp_impl(str, str3));
    printf("%d\n", strcmp_impl(str, str4));

    printf("%d\n", strcmp_impl(str1, str));
    printf("%d\n", strcmp_impl(str2, str));
    printf("%d\n", strcmp_impl(str3, str));
    printf("%d\n", strcmp_impl(str4, str));
}
```

Output:

```
-97
-1
-1
0

97
1
1
0
```

הפונקציה strtok

- אפשר להשתמש בפונקציה כדי לחלק מחרוזת לתתי מחרוזת.
- נדרשת הבנה במצביעים ,לכן רק נציג את השימוש בה.

```
#include <stdio.h>
#include <string.h>

void strtok_ex(char str[], char delim[]) {
    char* token = strtok(str, delim);
    while (token != NULL) {
       printf("%s\n", token);
       token = strtok(NULL, delim);
    }
}
```

הפונקציה strtok

```
void main () {
    char str[] = "Hello, my website is: www.google.com";
    strtok_ex(str, ",");
}
```

Output:

Hello

my website is: www.google.com

הפונקציה strtok

```
void main () {
    char str[] = "Hello, my website is: www.google.com";
    strtok_ex(str, ".");
}
```

Output:

Hello, my website is: www

google

com

הפונקציה strtok

```
void main () {
    char str[] = "Hello, my website is: www.google.com";
    strtok_ex(str, " ");
}
```

Output:

Hello,
my
website
is:
www.google.com

הפונקציה strtok

```
void main () {
   char str[] = "Hello, my website is: www.google.com";
   strtok_ex(str, ",. ");
}
```

Output:

Hello
my
website
is:
www
google
com

הפונקציה strtok

```
void main () {
    char str[] = "Hello, my website is: www.google.com";
    strtok_ex(str, ",. :");
}
```

Output:

Hello
my
website
is
www
google
com

מצביעים

הזיכרון

- הזיכרון מחולק לבתים כאשר לכל בית יש כתובת.
- כל משתנה תופס את גודלו בזיכרון בבתים.

1013	1012	1011	1010	1009	1008	1007	1006	1005	1004	1003	1002	1001	1000
int i								short s					char c

- קריאה וכתיבה למשתנים מתורגמת על ידי הקומפיילר לקריאה וכתיבה אל הכתובת של אותם משתנים בזיכרון.
- ניתן לקבל את כתובתו של משתנה על ידי שימוש באופרטור &.

מצביע

- מצביע הוא משתנה המכיל כתובת.

- כל הכתובות במערכת אחת הן באותו גודל.
  - במערכת 32bit כל הכתובות יהיו בגודל של 4 בתים.
  - במערכת 64bit כל הכתובות יהיו בגודל של 8 בתים.

- למרות שכל הכתובות זהות מבחינת הגודל, קיימת אבחנה בין כתובות של טיפוסים שונים.

- הגדרת מצביע נעשית באחד משני האופנים הבאים:

```
type* ptr-name;
type *ptr-name; // Recommended
```

מצביע

- שינוי הערך המוצבע יעשה באמצעות האופרטור :*

```
*ptr = value;
```

- אפשר להגדיר מצביע לכל טיפוס שכבר קיים במערכת.
  - מסקנה :עבשיו קיימים)תיאורטית (אינסוף טיפוסים.

```
int *p;
char **ptr;
```

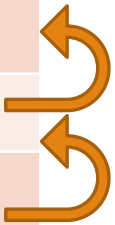
- כאשר מגדירים מצביעים ברשימה ,יש לכתוב *עבור כל אחד.

```
int *p1, *p2, *p3;
```

דוגמה

```
int main() {
    int x = 1;
    int *p1 = &x;
    int **p2 = &p1;
    printf("%d\n", x);
    *p1 = 2;
    printf("%d\n", x);
    **p2 = 3;
    printf("%d\n", x);
    x = *p1 + **p2;
    printf("%d\n", x);
}
```

value	identifier	address
1	x	1000
1000	p1	2000
2000	p2	3000



Output:

1

2

3

6

דוגמה

```
void main() {
    int x = 1;
    int *p = &x;
    printf("x  : %d\n", x);
    printf("p  : %p\n", p);
    printf("*p : %d\n", *p);
    printf("&x : %p\n", &x);
    printf("*&p: %p\n", *&p);
    printf("&*p: %p\n", &*p);
    printf("*&x: %d\n", *&x);
// printf("&*x: %d\n", &*x); // ← Can only do * on a pointer.
}
```

Output:

```
x   : 1
p   : 0x7fffe154773c
*p  : 1
&x  : 0x7fffe154773c
*&p: 0x7fffe154773c
&*p: 0x7fffe154773c
*&x: 1
```

אריתמטיקת מצביעים

- ישנם ארבעה אופרטורים מתמטיים שאפשר להפעיל על מצביע.

++, --, +, -

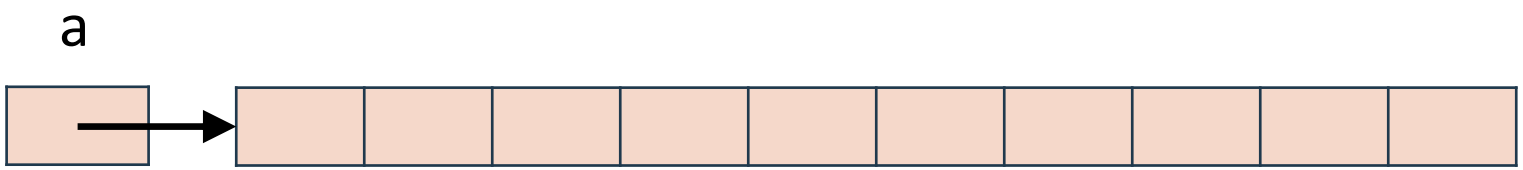
- האופרטורים פועלים באופן שונה על מצביעים מטיפוסים שונים.

New address	Calculation	Operator	Initial address	Pointer type
1001	1000 + sizeof(char)	p1++	1000	char *p1
2002	2000 + sizeof(short)	p2++	2000	short *p2
3004	3000 + sizeof(int)	p3++	3000	int *p3
4008	4000 + sizeof(long)	p4++	4000	long *p4

מערכים ומצביעים

- בהגדרת מערך מוגדרים שני דברים באופן אוטומטי.
  - **<u>שטח הזיכרון</u>** לאחסון איברי המערך.
  - **<u>מצביע קבוע</u>** שמאותחל בכתובתו של האיבר הראשון.
  - שם המערך הוא למעשה המצביע הזה.

```
type a[10];
```



- בשל אריתמטיקת מצביעים a+i, הוא מצביע לאיבר ה i-במערך.
  - כי בפועל החישוב הוא (a + i * sizeof(type

- האופרטור []מתורגם על ידי הקומפיילר באופן הבא:

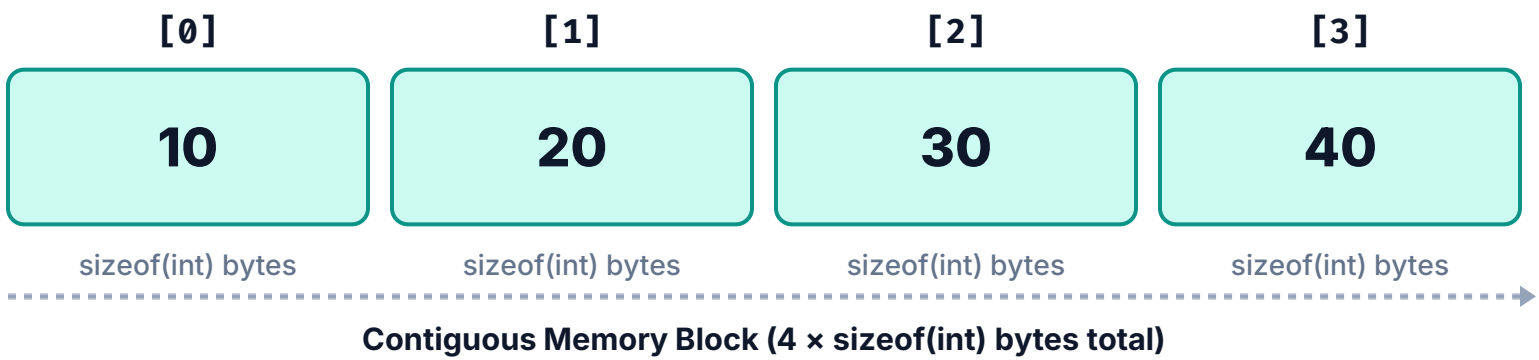
```
a[i] => *(a+i)
```

- לאור זאת, תכונות של מערכים מובנות יותר.
  - האינדקס הראשון במערך הוא 0.
  - מערך לא מועתק כשהוא נשלח לפונקציה, כי מעתיקים רק את המצביע.
  - אם ניגש למערך מחוץ לגבולות זה ניגש לכתובות שנמצאות אחרי המערך (overflow).

```
a[0] => *(a+0) => *a
```

Arrays in Memory

```
int values[4] = {10, 20, 30, 40};
```

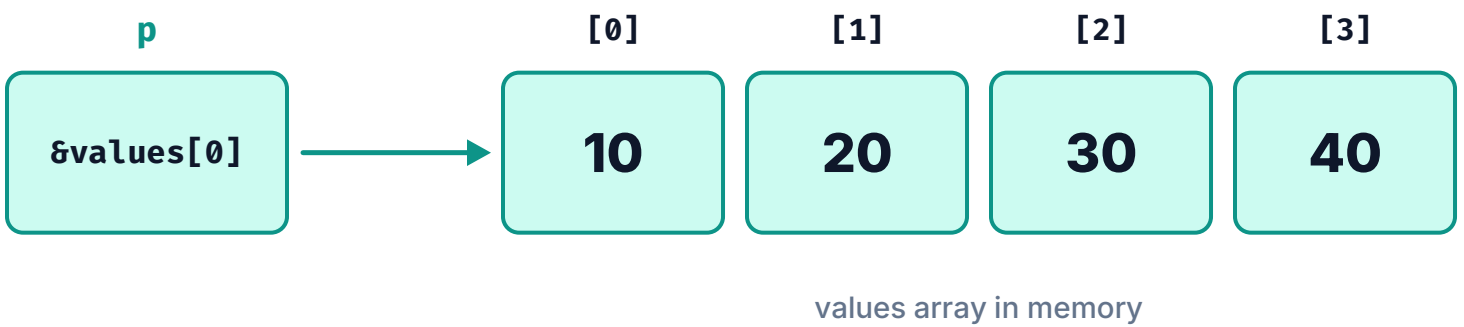


- Valid indices: **0 through 3**
- `sizeof(values) == 4 * sizeof(int)`

What happens if we read values[4]?

A First Look at Pointers

```
int values[4] = {10, 20, 30, 40};
int *p = &values[0];
```



p stores the address of values[0]

***p** reads the value 10

פרמטרים לשורת הפקודה argc ו־argv

- ניתן להעביר מידע לתוכנית בזמן ההפעלה דרך שורת הפקודה.
- argc — מספר הארגומנטים, כולל שם התוכנית.
- argv — מערך של מחרוזות הארגומנטים; argv[0] הוא בדרך כלל שם התוכנית או הנתיב שלה.

קוד

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for (int i = 0; i < argc; i++) {
    printf("argv[%d] = %s\n",
           i, argv[i]);
  }
  return 0;
}
```

הרצה ופלט

Command:

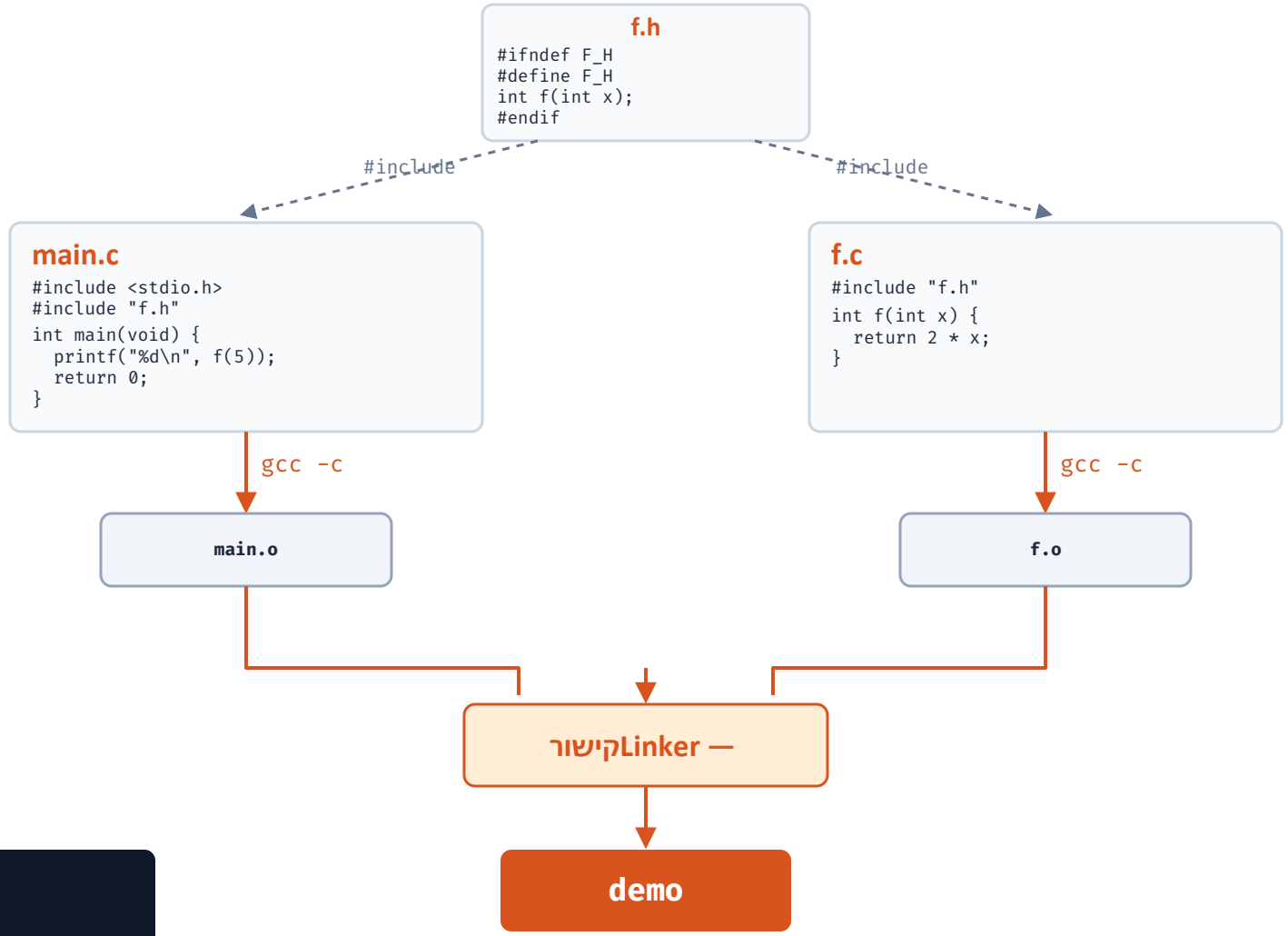
```
./demo Nour 23
```

Output:

```
argc = 3
argv[0] = ./demo
argv[1] = Nour
argv[2] = 23
```

💡 גם "23" מתקבל כמחרוזת, ולא כמספר שלם.

קומפילציה וקישור




```
$ ./demo
10
```

- **קומפילציה:** כל קובץ מקור מתורגם בנפרד לקובץ אובייקט.
- **קישור:** קובצי האובייקט מחוברים, והקריאה ל־f מקושרת למימוש שלה.